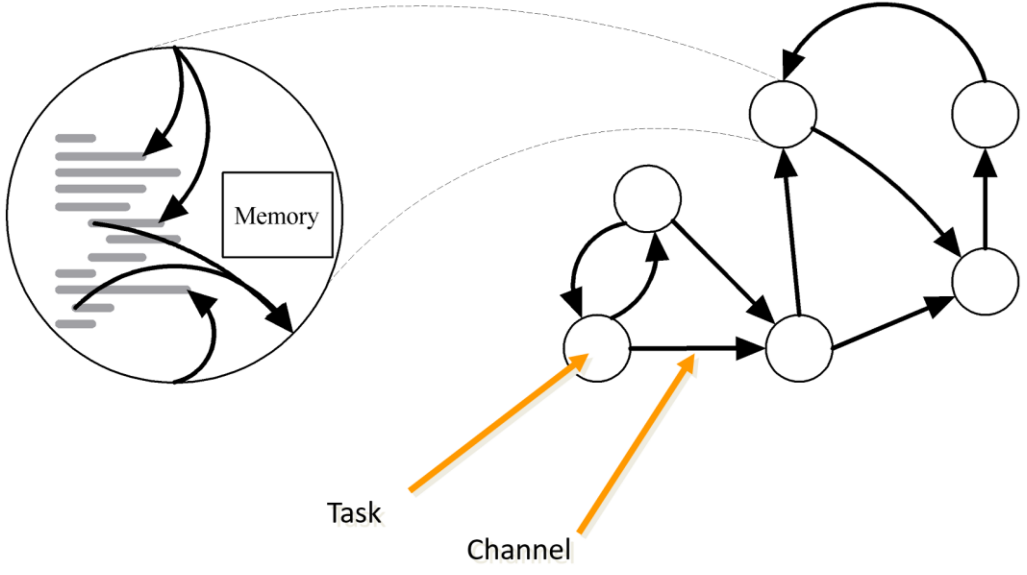
CPSC 375 High-Performance Computing

Lecture 15

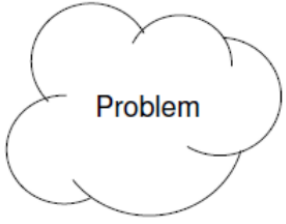
Parallel Algorithm Design

Task/Channel Model

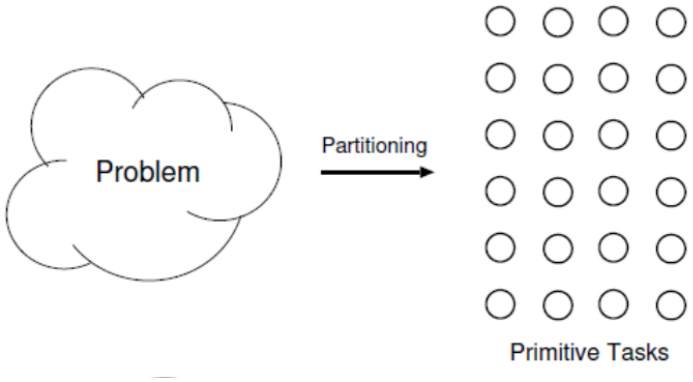
- Parallel computation = set of tasks
- Task
  - Program
  - Local memory
  - Collection of I/O ports
- Tasks interact by sending messages through *channels*



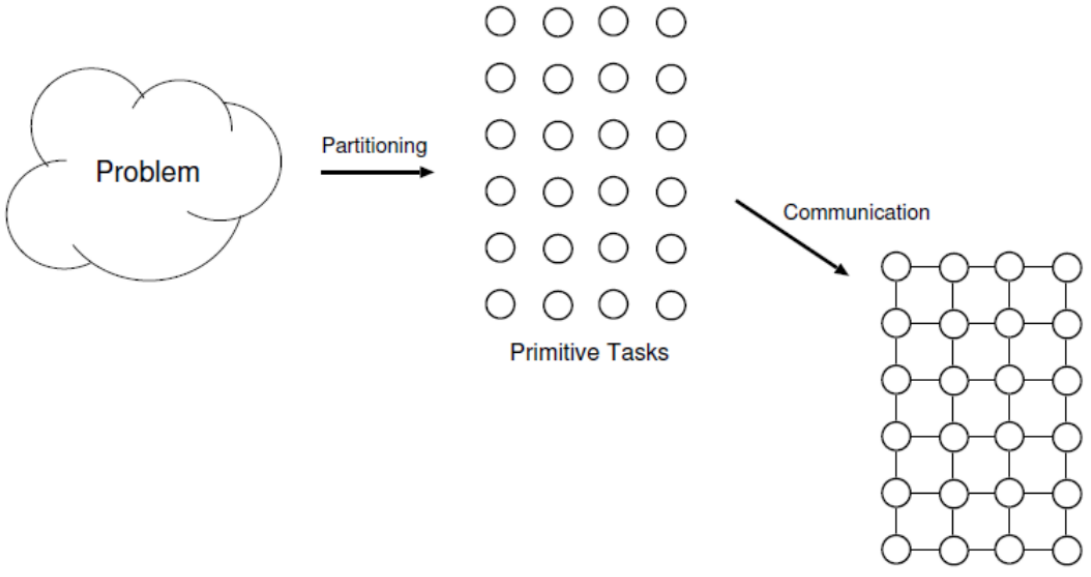
Foster's Design Methodology



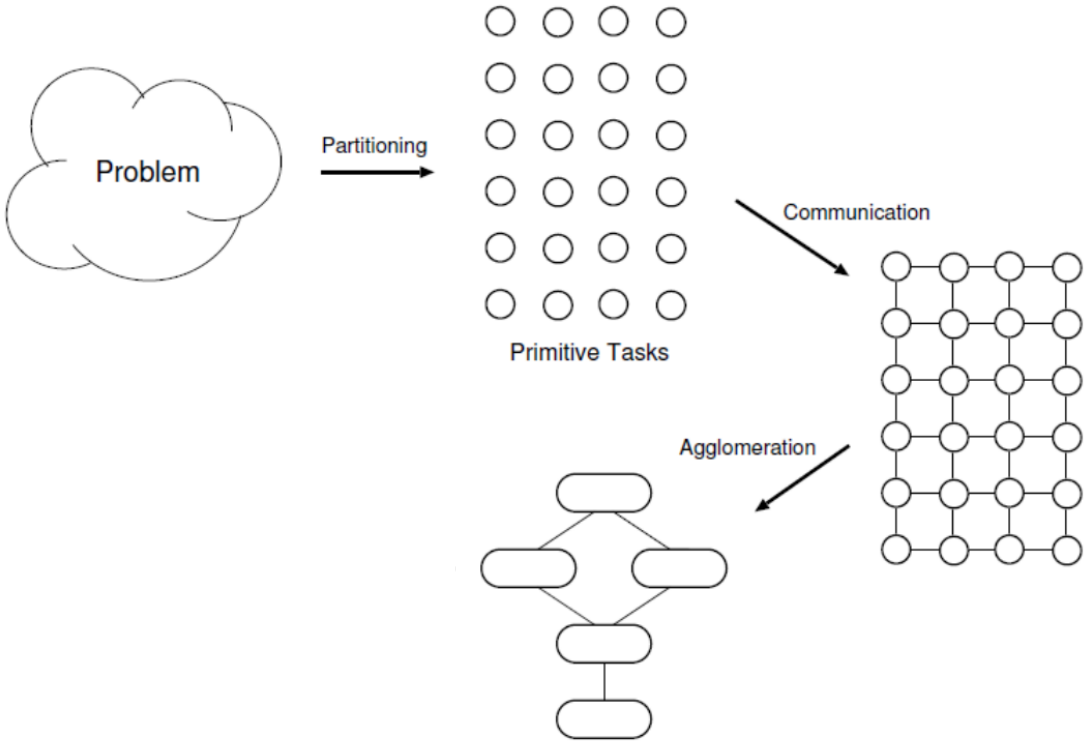
Foster's Design Methodology



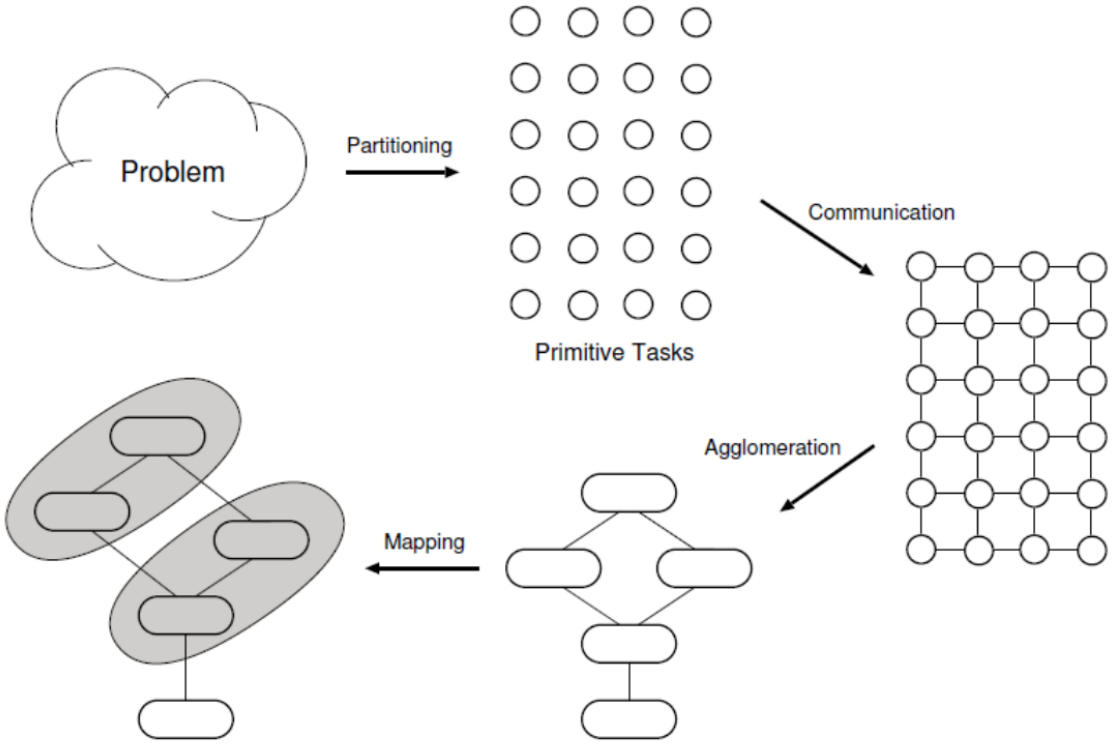
Foster's Design Methodology



Foster's Design Methodology



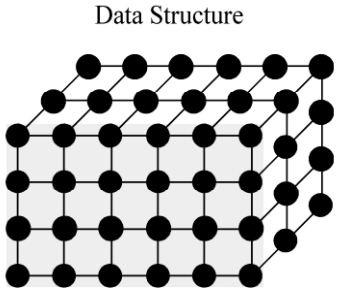
Foster's Design Methodology



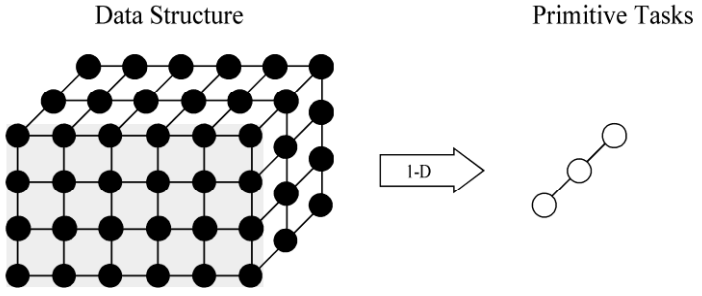
Partitioning

- Dividing computation and data into pieces
- *Domain decomposition*
  - Divide *data* into pieces
  - Determine how to associate computations with the data
- *Functional decomposition*
  - Divide *computation* into pieces
  - Determine how to associate data with the computations

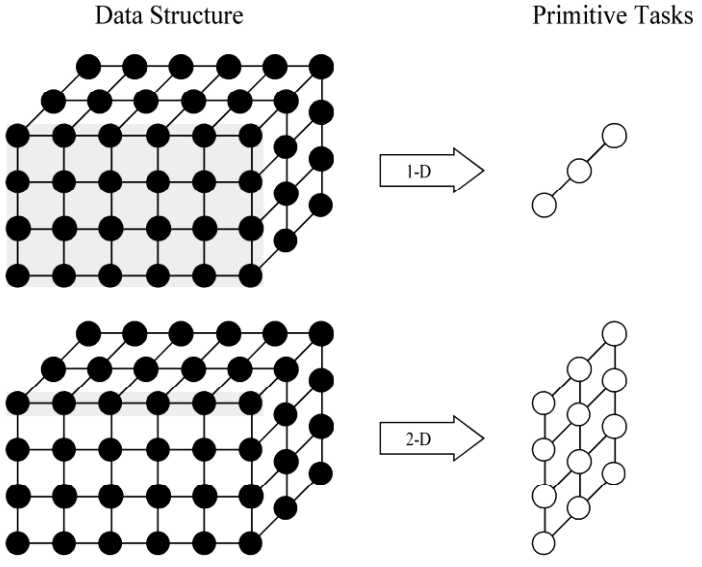
Example Domain Decompositions



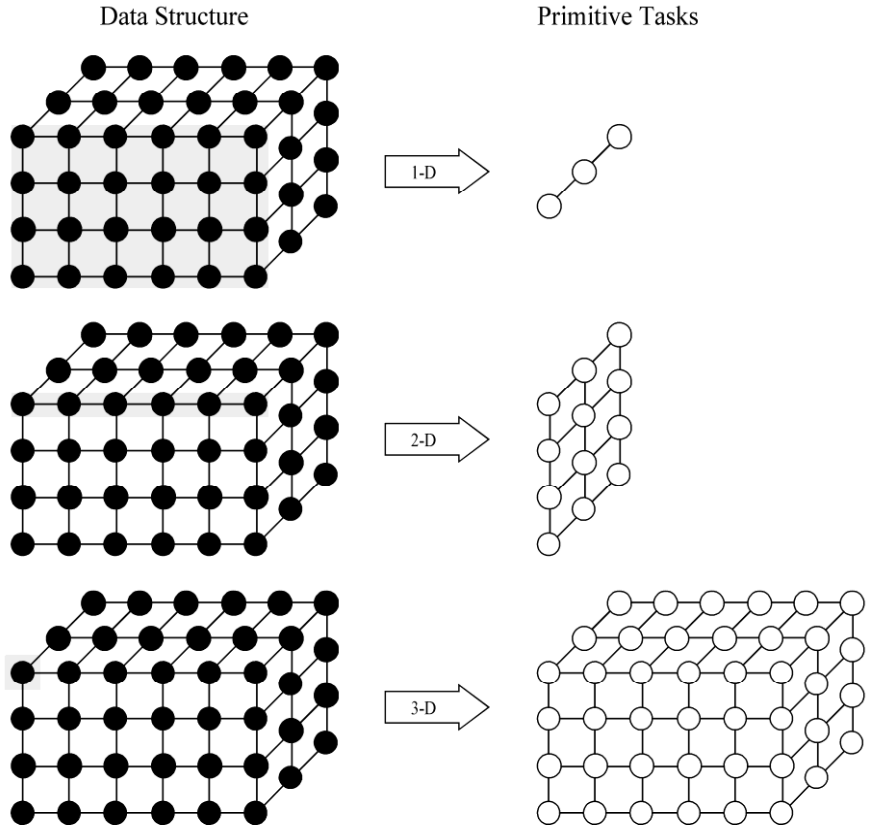
Example Domain Decompositions



Example Domain Decompositions



Example Domain Decompositions



Example Functional Decomposition

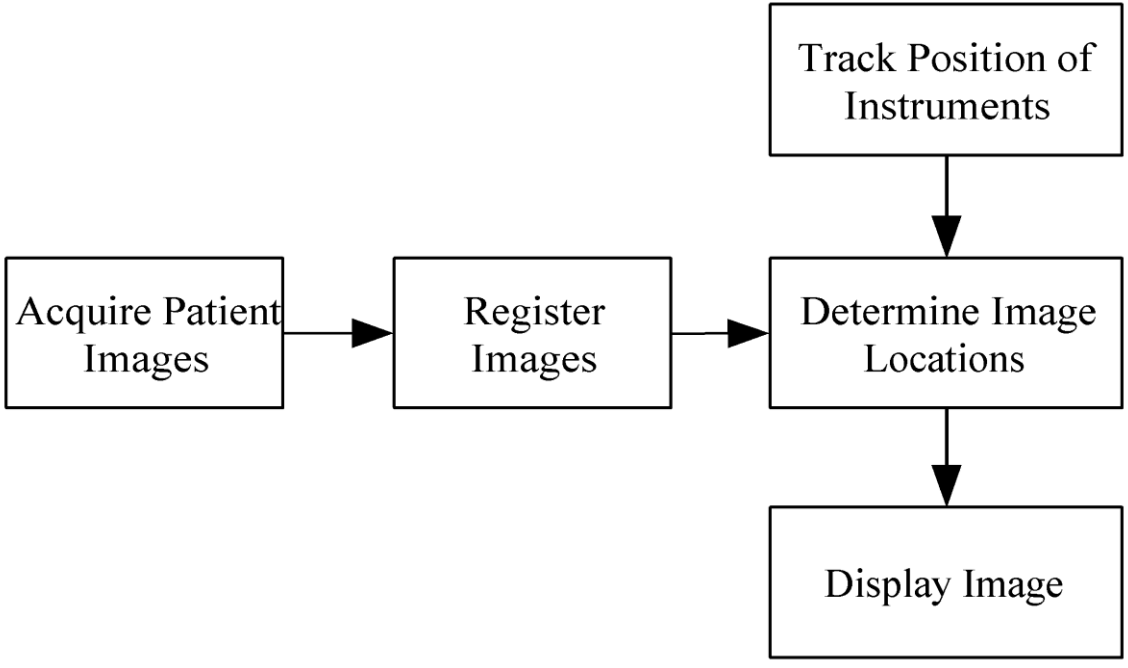


Image-guided brain surgery

Partitioning Checklist

- Identify as many *primitive* tasks as possible.
  - At least an order of magnitude more primitive tasks than processors in the target computer
- Minimize redundant computations and redundant data storage.
- Primitive tasks are roughly the same size.
- Number of tasks is an increasing function of problem size.

Communication

- *Local communication*
  - Task needs values from a small number of other tasks
  - Create channels illustrating data flow
- *Global communication*
  - Significant number of tasks contribute data to perform a computation
  - Don't create channels for them early in design
- Overhead of a parallel algorithm, as it is something the sequential algorithm does not need to do.

Communication Checklist

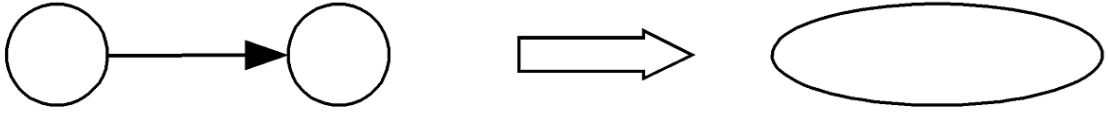
- Communication operations balanced among tasks
- Each task communicates with only small group of neighbors
- Tasks can perform communications concurrently
- Tasks can perform computations concurrently

Agglomeration

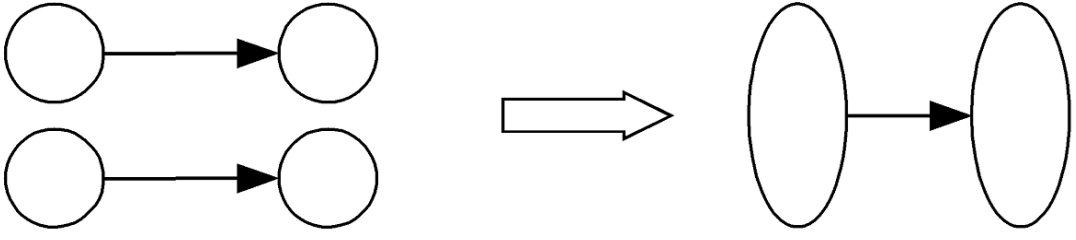
- Grouping tasks into larger tasks
- Goals
  - Lower communication overhead by increasing the *locality*, thus improving performance
  - Maintain the *scalability* of the program
  - Reduce software engineering cost
- When developing MPI programs, we create one agglomerated task per processor

Agglomeration Can Improve Performance

- Eliminate communication between primitive tasks agglomerated into consolidated task



- Combine groups of sending and receiving tasks



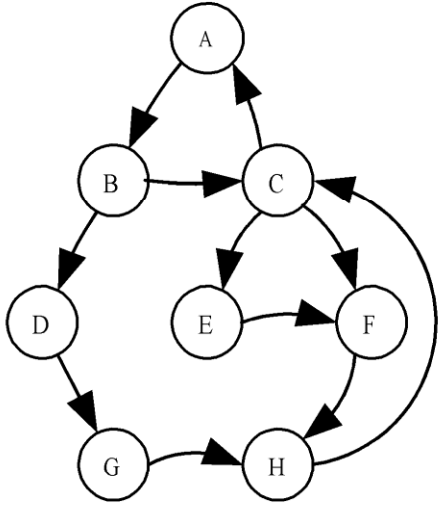
Agglomeration Checklist

- Locality of the parallel algorithm has increased
- Replicated computations take less time than communications they replace
- Data replication doesn't affect scalability
- Agglomerated tasks have similar computational and communications costs
- Number of tasks increases with problem size
- Number of tasks suitable for likely target systems
- Tradeoff between agglomeration and code modifications costs is reasonable

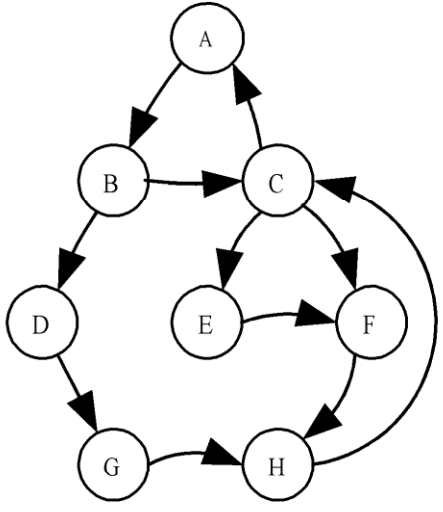
Mapping

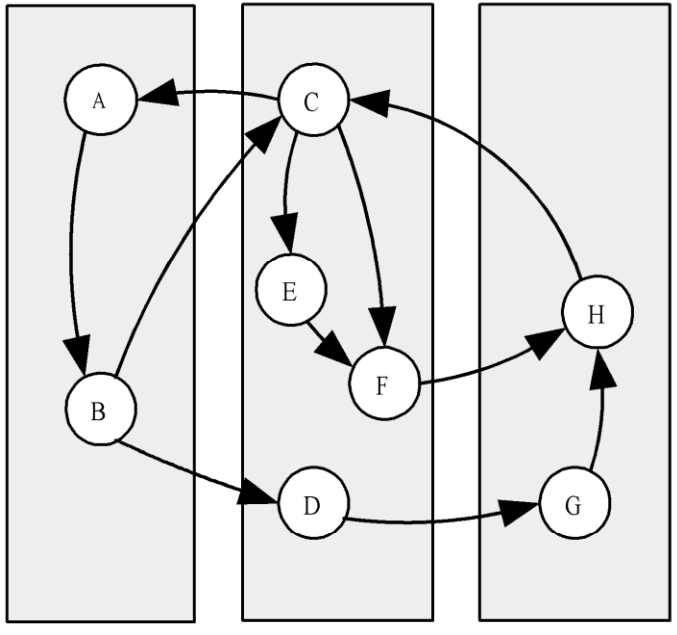
- Process of assigning tasks to processors
- *Centralized multiprocessor*: mapping done by operating system
- *Distributed memory system*: mapping done by user
- Conflicting goals of mapping
  - Maximize *processor utilization*, the average percentage of time the processors are actively executing tasks.
  - Minimize interprocessor communication

Mapping Example



Mapping Example





- Finding optimal mapping is NP-hard.
- Must rely on heuristics.

Mapping Decision Tree

- Static number of tasks
  - Structured communication
    - Constant computation time per task
      - Agglomerate tasks to minimize communication
      - Create one task per processor
    - Variable computation time per task
      - Cyclically map tasks to processors
  - Unstructured communication
      - Use a static *load balancing* algorithm
- Dynamic number of tasks

Mapping Strategy

- Dynamic number of tasks
  - Frequent communications between tasks
    - Use a dynamic load-balancing algorithm
  - Many short-lived tasks
    - Use a run-time task-scheduling algorithm

Static Load Balancing

- Distribution of tasks to processors determined a priori.
- Consistent workload per processor
- No need for monitoring, task migration, or communication
- Assumptions:
  - Tasks with roughly the same complexity
  - Processors with the same performance
- Inability to adapt to data-dependent branches

Dynamic Load Balancing

- Real-time workload monitoring, adjusting the assignment of tasks during execution.
- On-the-fly task migration, tracking processor utilization and queue length.
- Adaptive hardware responsiveness, considering processors with higher performance.
- Reactive data-dependent partitioning, responding to unpredictable changes in the computation.

Run-Time Task Scheduling

- Dynamically selects which ready task executes next based on current system state.
- Maintains a ready queue of tasks waiting for CPU allocation
- Applies a scheduling policy .
  - e.g., FCFS, Round Robin, Priority, etc.
- Supports preemption to allow higher-priority tasks to interrupt execution.
- Performs context switching to transfer CPU control between tasks.
- Optimizes performance metrics such as CPU utilization, response time, and fairness.

Mapping Checklist

- Considered designs based on one task per processor and multiple tasks per processor.
- Evaluated static and dynamic task allocation.
- If dynamic task allocation is chosen, task allocator is not a bottleneck to performance.
- If static task allocation is chosen, the ratio of tasks to processors is at least 10:1.